

Reporte de práctica P5: Estructuras de Selección

MPOO 2027-1 · Grupo 6 · Producto.java y Expendedora.java

Objetivo. Explicar los errores encontrados durante la implementación de la práctica, indicar por qué ocurrían, describir cómo se solucionaron y documentar el resultado final de las pruebas.

1. Contexto de la práctica

La práctica P5 trabaja con estructuras de selección en Java: **if**, **else if**, **else**, operador ternario y **switch**. La clase *Producto* representa un artículo de una máquina expendedora y controla su precio y existencia. La clase *Expendedora* utiliza objetos *Producto* para administrar artículos, saldo, compras y ventas.

2. Errores encontrados y soluciones

Error 1. No se asignaba la capacidad máxima

En el constructor de *Producto* se escribió `this.Capacidad;`. Esa instrucción únicamente hacía referencia al identificador y no modificaba el atributo `piezas`.

```
if (piezas > CAPACIDAD) { this.piezas = CAPACIDAD; }
```

Error 2. Condición incorrecta en `hayPiezas()`

Inicialmente se utilizó `piezas < 0`. Sin embargo, el constructor evita que las piezas sean negativas. El método debía responder `true` cuando quedara al menos una pieza.

```
boolean hayPiezas() { return piezas > 0; }
```

Error 3. Faltaba cerrar `sacarUna()`

Faltaba una llave de cierre después del `if` de `sacarUna()`. Como consecuencia, el siguiente bloque quedaba dentro del método anterior y la estructura de la clase era incorrecta.

```
void sacarUna() { if (piezas > 0) { this.piezas = piezas - 1; } }
```

Error 4. Se intentaba devolver texto desde un método `void`

Las condiciones que devolvían "AGOTADO", "ULTIMAS PIEZAS" y "DISPONIBLE" se habían colocado dentro de `sacarUna()`. Como `sacarUna()` es `void`, no puede devolver un `String`. Esa lógica pertenece al método `existencia()`.

```
String existencia() { if (piezas == 0) { return "AGOTADO"; } else if (piezas <= 2) { return "ULTIMAS PIEZAS"; } else { return "DISPONIBLE"; } }
```

Error 5. La condición excluía el caso de 2 piezas

Se había escrito `piezas < 2`. Después de descartar el caso cero, esa condición solamente reconocía una pieza. El enunciado establece que tanto 1 como 2 deben producir "ULTIMAS PIEZAS".

```
else if (piezas <= 2) { return "ULTIMAS PIEZAS"; }
```

Error 6. Convención del nombre de la constante

La constante se declaró inicialmente como `Capacidad`. Aunque Java puede aceptar ese nombre, el ejercicio y la convención de Java indican que las constantes `static final` deben escribirse en mayúsculas.

```
static final int CAPACIDAD = 10;
```

3. Resultado de Producto.java

Con las correcciones, Producto valida correctamente el inventario: valores negativos se convierten en 0, valores mayores que 10 se limitan a la capacidad máxima, sacarUna() nunca reduce la existencia por debajo de cero y existencia() clasifica correctamente el inventario.

4. Implementación de Expendedora.java

En Expendedora se utilizaron objetos Producto para representar papas, refresco y chocolate. También se agregó un contador static para las ventas de todas las máquinas, un saldo y una ubicación. El constructor crea los productos y utiliza un operador ternario para sustituir una ubicación en blanco por "SIN UBICACION".

```
static int ventasTotales = 0; String ubicacion; int saldo; Producto papas; Producto refresco;
Producto chocolate;
```

El método buscar() normaliza el código mediante trim() y toUpperCase() y utiliza un switch para relacionar A1 con papas, A2 con refresco y B1 con chocolate. Las operaciones de compra revisan primero que el código sea válido, después la existencia y finalmente el saldo antes de entregar el producto.

5. Ejecución de las pruebas

Después de completar y corregir las clases, se ejecutó P5Seleccion. La salida final mostró que las 18 pruebas coincidieron con los resultados esperados, incluyendo validación de piezas, estados de existencia, búsqueda de productos, compras, cambio, cancelación y conteo total de ventas.

```
$ java P5Seleccion
```

P5 · ESTRUCTURAS DE SELECCION				
#	prueba	esperado	obtuviste	
1	PRODUCTO			
	papas con 15 piezas	10	10	
	ok			
2	chicles con -4 piezas	0	0	
	ok			
3	chicles.existencia()	AGOTADO	AGOTADO	
	ok			
4	gomitas.existencia()	ULTIMAS PIEZAS	ULTIMAS PIEZAS	ok
5	papas.existencia()	DISPONIBLE	DISPONIBLE	
	ok			
6	gomitas: sacar 3 de 2	0	0	
	ok			
7	EXPENDEDORA			
	m.etiqueta()	Edificio Q (DISPONIBLE)	Edificio Q (DISPONIBLE)	ok
8	v.etiqueta()	SIN UBICACION (VACIA)	SIN UBICAC	
	(VACIA)			
9	buscar(" a2 ").getPrecio()	22	22	ok
10	Z9 no existe, b1 si	true, true	true, true	ok
11	\$15, comprar("A1")	FALTAN \$3	FALTAN \$3	ok
12	+\$3, comprar("A1")	ENTREGADO, CAMBIO \$0	ENTREGADO, CAMBIO \$0	ok
13	piezas de A1 despues	9	9	
	ok			
14	comprar("C3")	CODIGO INVALIDO	CODIGO INVALIDO	ok
15	\$30, comprar("B1")	ENTREGADO, CAMBIO \$15	ENTREGADO, CAMBIO \$15	ok
16	maquina vacia, comprar("B1")	AGOTADO	AGOTADO	ok
17	v.cancelar() dos veces	10, 0	10, 0	ok
18	getVentasTotales()	2	2	
	ok			
18 DE 18				
Listo. Toma la captura de esta tabla y pegala en tu bitacora.				

Figura 1. Resultado de P5Seleccion: 18 de 18 pruebas correctas.

6. Conclusiones

Los principales problemas fueron de asignación, comparación, delimitación de bloques y tipos de retorno. Al corregirlos se reforzó la diferencia entre consultar un atributo y asignarle un valor, el uso de expresiones booleanas, la importancia de cerrar correctamente cada método y la necesidad de respetar el tipo de retorno declarado. La ejecución final confirmó que las dos clases cumplen el comportamiento solicitado, ya que P5Seleccion reportó 18 de 18 pruebas correctas.